

Batch Normalization Processor Design for Convolution Neural Network Training and Inference

Yu-Sheng Ting*, Yu-Fan Teng*, and Tzi-Dar Chiueh

Department of Electrical Engineering, National Taiwan University, Taipei 10617, Taiwan

*equal contribution

Abstract—In the training process of convolutional neural networks (CNN), a batch normalization (BN) layer is often inserted after a convolution layer to accelerate the convergence of CNN training. In this work, we propose a BN processor that supports both the training and inference processes. To speed up the training of CNN, the proposed work develops an efficient dataflow integrating a novel BN processor design and the processing elements for convolution acceleration. We exploited the similarities in the calculations required for the BN forward and backward passes by sharing hardware elements between both passes, therefore reducing the area overhead. In addition to functional verification of the BN processor, we also completed Automatic Placement & Routing (APR) and conducted post-APR simulation on neural network training. Finally, the proposed solution not only significantly speeds up the CNN training process, but also achieves hardware saving.

Keywords—batch normalization, accelerator, hardware, training, convolutional neural network

I. INTRODUCTION

In recent years, CNN has been a widely-used NN structure in deep learning. Since a large amount of operations are needed for training and inference of CNNs, several hardware accelerators have been developed to speed up the convolution computation [1] [2] [3]. However, fewer works focused on accelerating non-convolutional layers, and as such those calculations are handled by software, resulting in undesirable timing overhead. Among the non-convolutional layers, BN layers [4] account for the most operations since they involve basic statistical calculation (mean and standard deviation) and many precise and large-dynamic-range divisions and multiplications. Despite the huge amount of computation, it is quite regular and data-intensive. As a result, BN layer is an obvious target for hardware acceleration.

Some hardware implementations based on FPGA have been proposed to accelerate the BN layer processing in CNN inference [5][6]. In these works, hardware overhead is reduced by combining BN with the ReLU activation function or the succeeding convolution layer, which is feasible because the statistical parameters are constant in the inference stage. However, such designs are not applicable in the training process since statistical parameters resulting from different batches are different. To the best of our knowledge, there are only two BN layer hardware implementations that support both the training and inference processes, while they have some drawbacks as described below. Shuang *et al.* [7] proposed an L1-norm BN with only

linear operations in both forward and backward processes during training. This work focuses on the BN hardware implementation but neglects the interaction between the BN processor and the convolution processor elements. Another work supporting both the training and inference processes was proposed by Yang *et al.* [8]. In this work, BN operations are integrated with the preceding convolution operations in a systolic array. However, the forward and backward BN operations were accelerated by separate hardware modules and did not leverage hardware resource sharing. In this work, we propose a hardware implementation of the BN layer for CNN training and inference. The proposed design makes full use of the hardware resources by sharing modules for BN operations in forward and backward stages.

II. BACKGROUND

Ioffe *et al.* proposed to insert a BN layer after the convolution layer to accelerate the convergence of CNN training [4]. Within a training batch, the normalization for an input channel is formulated as follows:

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)} \quad (1)$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2 \quad (2)$$

$$\hat{x}^{(i)} = \frac{x^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (3)$$

Let the batch size be m , and $x^{(i)}$ refers to the i^{th} input data of the current batch. Moreover, two learnable parameters, γ and β , are introduced in the BN layer to scale and shift the normalized values as shown in Eq. (4):

$$y^{(i)} = \gamma \hat{x}^{(i)} + \beta \quad (4)$$

In the training process, the gradient of the weights can be obtained from loss L through the following equations

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^m \frac{\partial L}{\partial y^{(i)}} \hat{x}^{(i)} \quad (5)$$

$$\frac{\partial L}{\partial \beta} = \sum_{i=1}^m \frac{\partial L}{\partial y^{(i)}} \quad (6)$$

According to [4], the gradient of the input data can be deduced through the chain rule:

$$\frac{\partial L}{\partial x^{(i)}} = \frac{\partial L}{\partial \hat{x}^{(i)}} \frac{1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial L}{\partial \sigma^2} \frac{2(x^{(i)} - \mu)}{m} + \frac{\partial L}{\partial \mu} \frac{1}{m} \quad (7)$$

According to [9], Eq. (7) can be formulated as:

$$\frac{\partial L}{\partial x^{(i)}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} \left(\frac{\partial L}{\partial y^{(i)}} - \frac{1}{m} \frac{\partial L}{\partial \beta} - \frac{\hat{x}^{(i)}}{m} \frac{\partial L}{\partial \gamma} \right) \quad (8)$$

In Eq. (8), the values of $\partial L / \partial \beta$ and $\partial L / \partial \gamma$ needed to be calculated first. The summations in Eq. (5) and Eq. (6) are similar to Eq. (2) and Eq. (1). Therefore, we used Eq. (8) to calculate the gradient of the input data. The hardware

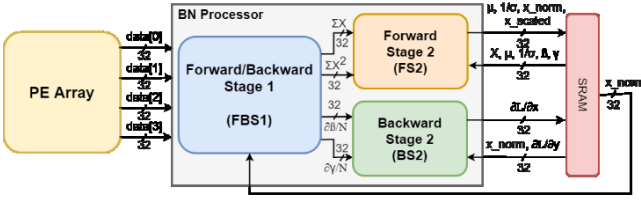


Fig. 1 Block diagram of the overall system architecture.

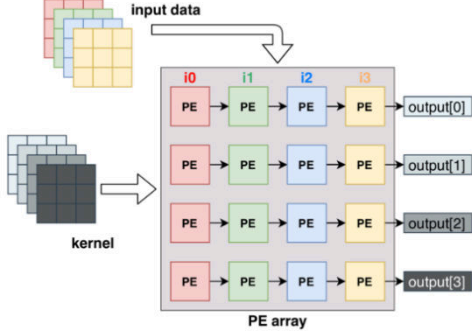


Fig. 2 Architecture of the (convolutional) PE array.

resources between the forward pass and the backward pass can hence be shared.

III. ARCHITECTURE

A. Top Design

The overall system architecture is shown in Fig. 1. It is composed of three parts: (convolutional) PE array, BN processor, and SRAM cache. The entire system works as follows. We assume that a BN layer is connected to the output of a convolution layer. During the forward process, the PE array calculates the convolution operation of the preceding layer's output. The results will then be sent to the BN processor to calculate mean and standard deviation as in Eq. (1) and Eq. (2). During the backward process, the same BN processor is responsible for calculating errors, gradients, and updating weights, as in Eqs. (5), (6), and (8). Note that the SRAM cache stores the intermediate results. In the forward and the backward processes, we used pipelined designs that make the PE array and the BN processor operate simultaneously to speed up CNN training.

B. Processing Element Array

The (convolutional) PE array is shown in Fig. 2, where each PE is responsible for calculating the multiplication and the summation of input data and weight. We extend a single PE to a 4x4 two-dimensional systolic array. Different PE rows calculate convolution of different output channels, while different PE columns handle different input channels. The results of PE on a row are accumulated to complete the convolution of a kernel. In this way, the PE array continuously outputs four convolution results of four independent output channels in every cycle. This process is repeated to ensuing rows, succeeding batches, and more output channels to complete the convolution layer operation.

C. BN Processor

The BN processor is divided into Forward/Backward Stage 1 (FBS1), Forward Stage 2 (FS2), and Backward Stage

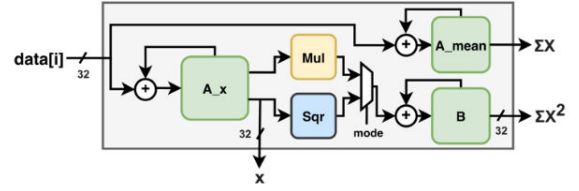


Fig. 3 Architecture of forward/backward stage 1.

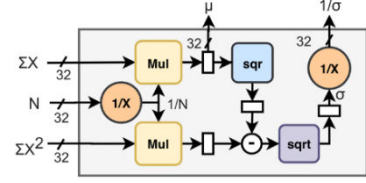


Fig. 4 Architecture of forward stage 2 for the statistical parameters.

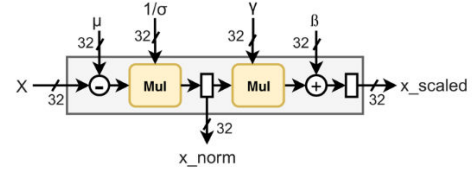


Fig. 5 Architecture of forward stage 2 for normalization and scaling.

2 (BS2). There are two-stage pipelined designs in the forward and the backward process, respectively. For example, FBS1 and FS2 are pipelined in the forward process, similarly for FBS1 and BS2 in the backward process.

(a) Forward Stage 1:

At this stage, the output values of the PE array are accumulated after the convolution operation and then sent to the next stage to calculate mean and standard deviation as Eq. (1) and Eq. (2). Fig. 3 shows one FBS1 module. There are four identical modules, and each processes the four outputs from the PE array. Each module consists of two 4x32-bit register files (A_mean and B), a 32x32 bit register file (A_x), a square module, a multiplier, and adders. At this stage, the PE array continues to perform the convolution operations according to Fig. 3. Then the values are fed to FBS1 and accumulated in A_mean and A_x. Also, the values in A_x will be fed to the SRAM, and then they will be squared and stored in B after accumulation (the multiplier will not be used in this stage). In our pipelined design, we use a ping-pong buffer to implement the process. When the accumulation is completed, the result will be sent to FS2, and the next input data will be fed to FBS1 to start a new convolution operation.

(b) Forward Stage 2:

At this stage, the statistical parameters (mean, standard deviation) of the data input in FBS1 are calculated as Eq. (3). Each data is normalized and scaled as Eq. (4) (multiplied by γ , and then incremented by β). The architecture of FS2 is divided into two parts: the first part (as shown in Fig. 4) is for the calculation of mean and standard deviation. The architecture consists of multipliers, a square module, a square root module, and a reciprocal module; the second part (shown in Fig. 5) is for calculation of normalization and scaling for the data sent from SRAM.

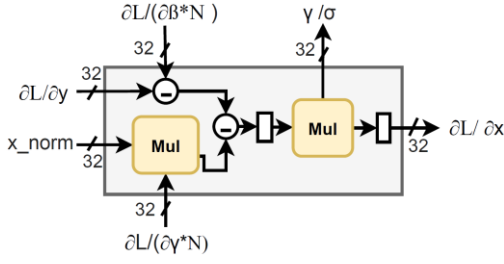


Fig. 6 Architecture of Backward Stage 2.

In the architecture of Fig. 4, we calculate the parameter N before calculating mean, where N is the size of the input data multiplied by the batch size (in the case of convolution, m in Eq. (1) should be replaced by N). It takes 10 cycles to do the reciprocal operation. Because N is fixed in the entire batch normalization layer, this method not only replaces the division required to calculate mean but also reduces the area required by the original divider. Another advantage is that it can complete the normalization of each data in one cycle. This method greatly reduces the calculation time required for the original division operation. After calculating the reciprocal of the average and standard deviation in Fig. 4, we move on to the calculation of normalization and scaling of the data. In Fig. 5, we use the mean and standard deviation to calculate the normalized data, and then we use γ and β for scaling. Finally, the results are then stored in the SRAM, completing the forward operation. At this stage, in order to reduce the cycle time, registers are inserted to buffer the input, output, and intermediate signals.

(c) Backward Stage 1

Our design utilizes the operation symmetry of forward and backward at stage 1 to design an architecture (FBS1) that can be shared by the operation of forward and backward passes, as shown in Fig. 3. At these two stages, the operations are similar except for the values in A_x . They will not be stored in the SRAM but will be multiplied, accumulated, and then stored in B (the square module will not be used at this stage). After the accumulation, the values in A_mean and B will be multiplied with the reciprocal of N before entering BS2. The purpose is the same as that of FS2. It can be used to replace the division required for the calculation of $\partial L/(\partial\beta*N)$ and $\partial L/(\partial\gamma*N)$ as Eq. (8).

(d) Backward Stage 2:

Since the normalized and scaled data have been calculated in FS2, the only thing left is to calculate the loss gradient values at the input. As shown in Fig. 6, this architecture is composed of a multiplier and a subtractor. The calculated $\partial L/\partial\gamma$, normalized scaling data, and the value calculated by FBS1 are obtained from SRAM. Finally, the gradient values are stored in the SRAM. Same as FS2, pipeline registers are inserted to reduce the cycle time.

The entire operation of the convolution layer and the BN layer can be executed in parallel. Since the operations of different output channels are independent, the calculations can be directly distributed into different PE arrays and BN processors, as shown in Fig. 7.

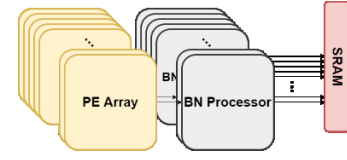


Fig. 7 Processors in parallel execution.

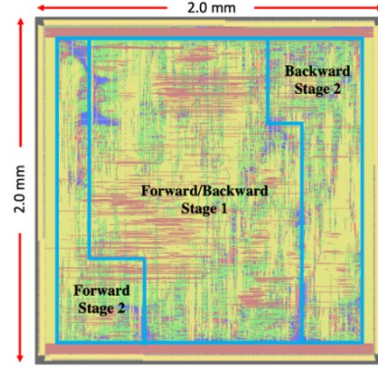


Fig. 8 Layout of BN acceleration IP.

IV. EVALUATION

We implemented our proposed design using a CMOS process to evaluate its performance. We conducted a post-APR simulation on the BN layers in a CNN of MNIST handwritten digit recognition. This section will focus on the analysis in terms of the execution time and chip area.

A. APR result

Fig. 8 shows the layout of our BN processor IP design and the delineation of three stages in the architecture. Table 1 shows the chip summary.

Table 1. Chip Summary

Technology	UMC 0.18um CMOS 1.8V/3.3V
IP size (μm^2)	2000.12 x 1999.76
Gate Count	1.41M
Power Supply	1.8V
Frequency	100MHz
Power	140.87mW

B. Timing analysis

As mentioned in Sec. III.B, the operation in the forward process will be iterated through the dimension of the input channel, the feature map, and the batch. Note that the 4x4 PE array can simultaneously calculates the convolution operations of four input channels. Accordingly, the number of cycles spent on stage 1 is

$$B \times M \times \left\lceil \frac{N_{input\ channel\ (pre-conv)}}{4} \right\rceil, \quad (9)$$

where B be the batch size, M be the size of the input feature map, and $N_{input\ channel\ (pre-conv)}$ refers to the number of the input channels in the preceding convolutional layer. Since the BN processor needs to complete the calculation of the four output channels, the number of cycles spent in Stage 2 is shown in Eq. (10):

$$B \times M \times 4 \quad (10)$$

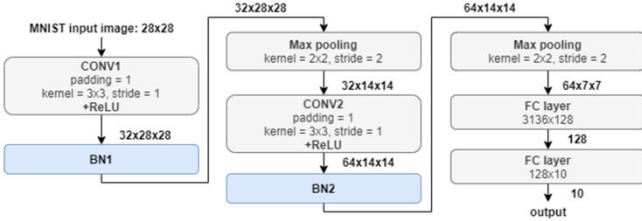


Fig. 9 Neural network of handwritten digit recognition.

Table 2. Comparison of the execution time

Execution time (μ s)	Intel® Core™ i7-6500U CPU	NVIDIA GeForce GTX 1080 GPU	Our design
BN1 Forward	2685.4	80.7	31.4
BN1 Backward	5695.1	209.7	62.72
BN2 Forward	796.5	83.9	15.68
BN2 Backward	12170.0	350.1	125.44

According to Eq. (9) and Eq. (10), during the forward process, the BN processor needs to wait for the PE array to calculate the convolution results if $N_{input\ channel\ (pre-conv)}$ is greater than 16. Due to the operation symmetry between the forward and backward passes, a similar conclusion can be deduced. In the backward process, the BN processor needs to wait for the PE array if the number of output channels of the next layer is greater than 16.

In most CNN, the number of input channels or output channels will be greater than 16. In other words, the BN processor completes the calculation before the PE array does in most cases. Therefore, BN operations can be completed without increasing the calculation time required for the original convolution.

We tested the executed time with real data, which were generated through the neural network of MNIST handwritten digit recognition as shown in Fig. 9. This network contains two BN layers, namely BN1 and BN2, both of which are inserted after two convolutional layers. The batch size for training is set to 16. In the test case, 16 BN processors operated in parallel. We compared the execution time with CPU and GPU as described in Table 2.

It is clear that our design outperforms GPU by more than 2 times in terms of the execution time, indicating the design can significantly save the training time of CNN training and inference.

C. Area

Since we share the hardware resources between forward pass and backward pass (FBS1), the chip area is therefore reduced. The gate count of FBS1 is 945.2K; that of FS2 and BS2 is 112.4K and 352.4K, respectively. We can deduce the area reduction ratio by Eq. (11), where G_{FBS1} , G_{FS2} , and G_{BS2} refer to the gate counts of FBS1, FS2, and BS2, respectively. The denominator means the gate count without hardware resource sharing, and the numerator means that of the shared design. Thus, the proposed architecture can reduce the total area by 40.13%.

$$\frac{G_{FBS1}}{(G_{FBS1} + G_{FS2}) + (G_{FBS1} + G_{BS2})} \quad (11)$$

D. Comparison

Among the aforementioned works, only L1-Norm BN [7] and Bactran [8] could support both inference and training processes. Our design presents a BN processor that interacts closely with the (convolutional) PE array, while no such consideration is mentioned in [7]. As a result, our design is more amenable to the actual circuit implemented for CNN training/inference acceleration. In addition, unlike [8], the innovative architecture that shares the hardware resources for forward and backward operations achieves significant reduction in required circuit complexity.

V. CONCLUSION

The new BN processor proposed in this work effectively reduces the time required for training of the deep convolutional network by interacting seamlessly with an existing convolution PE array. In this design, we calculate the forward and backward BN operations using the same architecture, resulting in higher hardware utilization. In order to further confirm the feasibility, we also implemented the BN processor in a CMOS process. Due to huge amount of calculation in deep learning applications, the use of special hardware to complete complex deep learning training tasks is indispensable. We believe the proposed BN processor realization can significantly facilitate the efficiency of convolutional neural network training as well as inference, and it will prove to be a valuable addition to NN acceleration hardware.

REFERENCES

- [1] C. Zhang, *et al.*, "Optimizing fpga-based accelerator design for deep convolutional neural networks," in *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*, 2015. p. 161-170.
- [2] Y. Chen, *et al.*, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, 2016, 52.1: 127-138.
- [3] M. Alwani, *et al.*, "Fused-layer CNN accelerators," in *Proc. of Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016. p. 1-12.
- [4] S. Ioffe, and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv preprint arXiv:1502.03167*, 2015.
- [5] Y. Wai, *et al.*, "Fixed point implementation of tiny-yolo-v2 using opencl on fpga," *International Journal of Advanced Computer Science and Applications*, 2018, 9.10: 506-512.
- [6] J. Ge, *et al.*, "BNReLU: Combine Batch Normalization and Rectified Linear Unit to Reduce Hardware Overhead. in *Proc. of International Conference on ASIC (ASICON)*, 2019. p. 1-4.
- [7] S. Wu, *et al.*, "L1-Norm batch normalization for efficient training of deep neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, 2018, 30.7: 2043-2051.
- [8] Z. Yang, *et al.*, "Bactran: A Hardware Batch Normalization Implementation for CNN Training Engine," *IEEE Embedded Systems Letters*, 2020.
- [9] W. Jung *et al.*, "Restructuring batch normalization to accelerate CNN training," *arXiv preprint arXiv:1807.01702*, 2018.